

1 Bus bunching prediction via early warning signals 2 in a transit digital twin

3 **Carlos Stiven Mora Hoyos¹, Juan Felipe Rodríguez Galindo¹, and Octavio**
4 **José Salcedo Parra¹**

5 ¹**Facultad de Ingeniería, Maestría en Ciencias de la Información y las Comunicaciones,**
6 **Universidad Distrital Francisco José de Caldas, Bogotá, Colombia**

7 Corresponding author:

8 Carlos Stiven Mora Hoyos¹

9 Email address: csmorah@udistrital.edu.co

10 ABSTRACT

11 Bus bunching is a stochastic operational anomaly that severely impairs the reliability of high-frequency
12 public transit corridors. Although the integration of General Transit Feed Specification Realtime
13 (GTFS-RT) data into Digital Twin (DT) architectures has been widely adopted, existing predictive
14 models frequently exhibit high false-positive rates due to urban signal noise, or fail to provide suffi-
15 cient lead time for operational intervention. This paper proposes an empirical Early Warning Signal
16 (EWS) algorithmic framework centered on computational efficiency and sequential validation. Using
17 a dataset of ~ 100 million raw Automatic Vehicle Location (AVL) records from San Francisco Muni
18 (January–March 2026)—strictly cleaned to 3,365,543 lag-valid observations for sequential modeling—
19 we demonstrate that a multi-stop sequential window ($n-3$) enriched with headway ratios, leader-bus
20 differential delay, and scheduling context effectively filters transient noise. While previous literature
21 often reports inflated metrics ($F1 > 0.80$) due to spatial-temporal data leakage and static anomaly
22 thresholds, we establish a rigorous evaluation regime using strict temporal hold-out and dynamic
23 bunching definitions ($h_{obs} < 0.25 \cdot h_{sch}$). Our results show that a LightGBM gradient boosting model
24 with physics-informed ratio features achieves an operational precision of 0.92 (95% CI: [0.91, 0.92])
25 and an F1-score of 0.56 (95% CI: [0.55, 0.56]) for predicting imminent bunching within a 3-stop hori-
26 zon. Crucially, we identify the headway ratio (h_{obs}/h_{sch}) and the relative delay differential between
27 consecutive buses as the primary precursor signals, confirming the cascade mechanism of bunching
28 propagation. Additionally, we report an inference latency of 2.15 milliseconds per record, enabling
29 real-time monitoring at regional scale. These spatio-temporal filtering and state-transition modeling
30 results inform proactive transit management in mixed-traffic environments and establish the basis for
31 control strategies in BRT systems.

32 1 INTRODUCTION

33 Bus bunching constitutes one of the primary operational failures in high-frequency public transit
34 networks (Daganzo, 2009; Yang et al., 2024). This phenomenon occurs when the temporal
35 separation between consecutive vehicles on the same line—referred to as headway—progressively
36 decreases until two or more buses travel in convoy. Daganzo (2009) formalized the causal
37 mechanism: when a bus accumulates a marginal delay at a stop, passenger demand at subsequent
38 stops increases, amplifying the original delay; simultaneously, the following vehicle encounters
39 fewer passengers and accelerates its journey, reducing the gap between them. Bartholdi and
40 Eisenstein (2012) demonstrated that this positive feedback is inherent to high-frequency lines
41 and grows exponentially without active control mechanisms. As a consequence, service degrades
42 in the form of excessive waiting times for passengers and unequal load distribution among
43 vehicles (Deng et al., 2020).

44 In practice, most transit agencies manage their fleets using static schedules defined under
45 the General Transit Feed Specification (GTFS) standard. These schedules establish planned
46 arrival times at each stop but lack predictive capacity to anticipate bunching events. Yang et al.
47 (2024) note that the operational detection of bunching is predominantly reactive: control centers

identify the event only after it has already manifested, limiting the possibilities for preventive intervention.

In parallel, Digital Twin (DT) architectures have emerged as a paradigm for proactive fleet monitoring and control (Botín-Sanabria et al., 2026; Manandhar et al., 2025). A transit DT constructs a virtual replica of the physical fleet from Automatic Vehicle Location (AVL) data transmitted via GTFS Realtime (GTFS-RT) (Hu et al., 2025). Nevertheless, the availability of real-time telemetry does not by itself resolve the prediction problem. Existing predictive frameworks face two documented limitations: high false-positive rates originating from the stochastic variance of urban traffic (signal cycles, double-parking, localized congestion), and computational overhead that prevents latencies compatible with real-time operational intervention (Kumar and Vanajakshi, 2024; Zhao et al., 2020).

This paper proposes a method to predict bus bunching before it occurs, with sufficient lead time for operators to intervene. Whereas conventional methods analyze what occurs at a single stop (Kumar and Vanajakshi, 2024), recent literature has demonstrated that incorporating information from multiple preceding stops improves the ability to distinguish transient disturbances from genuine deterioration trends (Deng et al., 2020; Boudabbous et al., 2026). Following this line, our approach examines the bus trajectory at the three preceding stops ($n-3$) alongside physics-informed ratio features—headway ratio (h_{obs}/h_{sch}), delay ratio, and leader-bus differential delay—that encode the cascade mechanism of bunching propagation. For classification, we expose the limitations of linear models on highly imbalanced, noise-heavy data, and employ a LightGBM gradient boosting model with class-imbalance weighting. Furthermore, we critique prior studies that report F1 scores above 0.80 by demonstrating how static thresholds (e.g., 120s fixed gaps) and random 80/20 data splits introduce severe data leakage and artificially inflate the positive class. By enforcing strict temporal hold-out and the theoretically robust dynamic bunching definition ($h_{obs} < 0.25 \cdot h_{sch}$), we achieve operationally reliable predictions (Precision = 0.92, F1 = 0.56) within a 3-stop horizon. Additionally, we incorporate conformal prediction—a statistical technique that, rather than issuing binary alerts, attaches to each prediction a set of possible labels with a calibrated coverage guarantee (e.g., 90%), enabling the operator to distinguish high-confidence alerts from ambiguous cases. Validation is performed on a San Francisco Municipal Railway (SF Muni) dataset processed with DuckDB (Raasveldt and Mühleisen, 2019) as the embedded analytical engine.

Concretely, the contributions of this work are:

- A physics-informed feature space ($n-3$, 24 features) that incorporates headway ratios (h_{obs}/h_{sch}), leader-bus differential delay, scheduled frequency, and spatial progression to capture the cascade mechanism of bunching propagation.
- A methodological critique establishing that standard cross-validation (e.g., 80/20) and fixed temporal thresholds induce spatial and temporal leakage, leading to artificially inflated EWS metrics ($F1 > 0.80$).
- A rigorous temporal hold-out evaluation demonstrating that under the dynamic definition of bunching ($h_{obs} < 0.25 \cdot h_{sch}$), a LightGBM gradient boosting model achieves operationally reliable precision (0.92) and F1-score (0.56).
- A conformal prediction layer that attaches calibrated confidence sets to each alert, so that operators can identify high-certainty alerts versus ambiguous situations requiring verification.
- A computational benchmark demonstrating low-millisecond latency using DuckDB, viable for deployment in low-latency Edge-Cloud Digital Twin architectures.
- Open code and data for full reproducibility.

The paper is organized as follows: Section 2 reviews the state of the art. Section 3 describes the system architecture and mathematical formulation. Section 4 details the experimental setup. Section 5 presents the results. Section 6 discusses the implications and limitations. Section 7 outlines future directions, and Section 8 concludes.

2 RELATED WORK

To construct a robust EWS framework, this study integrates literature from four domains: headway dynamics, data ingestion architectures, predictive modeling with imbalanced data, and Digital Twin frameworks.

2.1 Headway Dynamics and the Physics of Bus Bunching

The instability of high-frequency lines was mathematically formalized by Daganzo (2009), who showed that headway variance grows exponentially without active control. Bartholdi and Eisenstein (2012) extended this line by proposing self-coordinated headway-based strategies that replace static schedules with dynamic headway equalization. These foundational works establish that the relative distance between consecutive buses (headway) is a superior stability metric compared to schedule adherence (delay). Recent studies by Deng et al. (2020) and Kumar and Vanajakshi (2024) model bunching as a state-transition problem, using AVL and GTFS-RT data to track how local disturbances propagate spatially across the urban network. Olvera-Toscano et al. (2023) demonstrated that holding times based on quasi-regular headways reduce bunching penalties by up to 45%, provided the detection mechanism is accurate.

2.2 General Transit Feed Specification Realtime Data and Ingestion Architectures

The proliferation of open transit data has enabled large-scale operational analyses. Aemmer et al. (2022) proposed methodologies for extracting performance metrics from GTFS-RT components and classifying systematic versus stochastic delays. However, the large volume of AVL data imposes significant processing challenges (Ge et al., 2021). Traditional row-oriented relational databases (RDBMS) introduce substantial latency in the computation of sequential window functions (e.g., LAG operations over millions of rows) (Boncz et al., 2005). To address this, recent studies promote embedded column-oriented analytical databases. Raasveldt and Mühleisen (2019) introduced DuckDB as a highly optimized in-process database capable of executing complex analytical queries without network overhead. The application of these architectures to city-scale delay prediction has been validated by Boudabbous et al. (2026), emphasizing the need for multi-resolution feature engineering with minimal latency.

2.3 Machine Learning for Imbalanced Transit Data

Predicting bus bunching is inherently an imbalanced classification problem, since bunching events constitute the minority class (typically 5% to 10% of observations) under nominal operation (Al-Fares et al., 2024). Standard threshold classifiers or models optimized by global accuracy do not adequately capture the minority class, generating severe under-prediction of anomalies. Chen and Cheng (2023) demonstrated that imbalance-handling approaches focusing on F1-Score and Precision-Recall area under the curve (PR-AUC) are mandatory to avoid estimation bias in transportation models. Furthermore, Li et al. (2022) showed that addressing imbalance is critical in stochastic prediction of transportation anomalies. Although deep learning models such as the BAT-transformer (Jeong et al., 2023) achieve high accuracy in arrival time prediction, they operate as “black boxes” and require high computational resources. In contrast, optimized logistic regression provides interpretability for dispatchers while maintaining competitive predictive power when spatio-temporal features are adequately engineered (Yin et al., 2020). Yang et al. (2024) offer a comprehensive review indicating that, despite the popularity of deep learning, linear models with robust feature spaces tend to be more practical for real-time deployment.

2.4 Early Warning Systems in Public Transit

Early Warning Systems (EWS) constitute a specific class of predictive mechanisms oriented toward signaling the imminent onset of an anomaly before it fully manifests, distinguishing themselves from reactive detectors that operate a posteriori (Chandola et al., 2009). In the public transit domain, operational anomaly detection has historically been addressed through statistical control charts (CUSUM), adaptive threshold headway deviation models, and Gaussian process methods over punctuality time series (Kumar and Vanajakshi, 2024). These approaches generate alerts when a metric has already exceeded a predefined threshold, implying insufficient lead times for preventive mitigation actions.

151 Anomaly classifiers—including unsupervised methods (Isolation Forests), semi-supervised
 152 detectors (One-Class SVM), and supervised state-transition models—have emerged as predictive
 153 alternatives, enabling detection of deterioration signatures before observable collapse (Deng
 154 et al., 2020). Nevertheless, applying them in urban mixed-traffic corridors faces the challenge of
 155 severe class imbalance (anomaly events $\ll$ 10% of nominal observations) and the high exogenous
 156 variance of shared corridors (Al-Fares et al., 2024). Deep prediction models (transformers (Jeong
 157 et al., 2023), memory cell networks) achieve high accuracy in arrival times but operate as black
 158 boxes with latencies incompatible with real-time DT monitoring (Wang et al., 2024).

159 The framework proposed in this paper is positioned at the intersection of predictive EWS
 160 and low-latency DT architecture: we adopt a state-transition prediction formalism ($B_{n-1} = 0 \rightarrow$
 161 $\bar{B}_n = 1$, formalized in Section 3) to eliminate the persistence bias present in reactive detectors,
 162 and we wrap alerts with conformal prediction to provide calibrated probabilistic guarantees
 163 instead of unquantified binary alarms.

164 2.5 Digital Twin Architectures and Edge Computing

165 A transit Digital Twin integrates real-time telemetry with predictive models to construct a
 166 virtual replica of the physical fleet (Lu et al., 2020; Hossain and Karim, 2024). Fuller et al.
 167 (2020) describe structural requirements for sustainable public transportation DTs, highlighting
 168 “what-if” simulation capabilities. However, these capabilities depend on the generation of EWS
 169 with low latency. Bhatt et al. (2025) propose DT architectures for Intelligent Transportation
 170 Systems (ITS) that demand strict computational efficiency for continuous synchronization be-
 171 tween physical and virtual states. Consequently, edge computing paradigms are identified as
 172 key enablers of low-latency DTs (He et al., 2024). By processing data close to the source or
 173 using highly efficient in-process analytical tools, agencies can avoid the bottlenecks of central-
 174 ized cloud architectures (Chen, 2025). This hybrid Edge-Cloud architecture is consistent with
 175 resilience and security requirements observed in IoT-enabled Digital Twin deployments (Ahmed
 176 and Alnatheer, 2025).

177 3 SYSTEM ARCHITECTURE AND METHODOLOGY

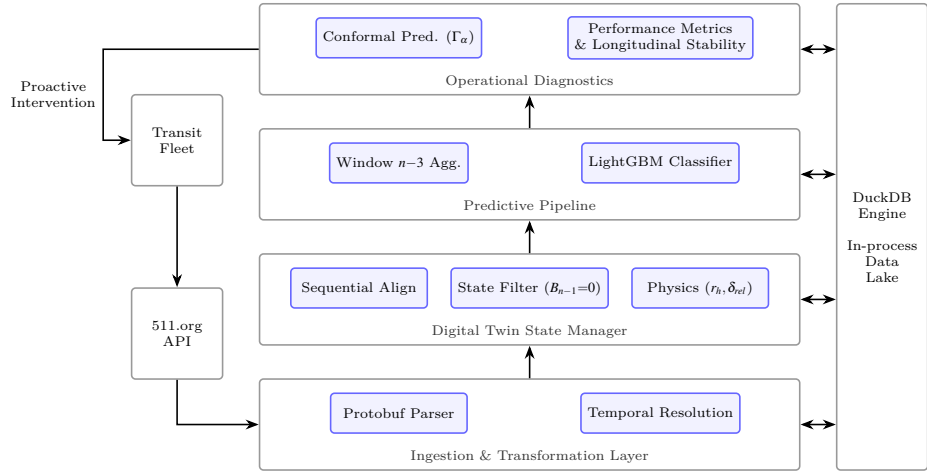


Figure 1. Proposed Early Warning Signal (EWS) architecture. The Physical layer provides real-time GTFS-RT telemetry. The Digital Twin layer, powered by DuckDB, manages state synchronization and $n-3$ feature engineering. The Predictive Intelligence layer executes state-transition modeling and conformal prediction to generate calibrated alerts.

178 3.1 Digital Twin Data Pipeline

179 The proposed system functions as the predictive layer of a Transit Digital Twin, following the
 180 multi-layered architecture illustrated in Fig. 1. Its objective is to receive real-time bus position
 181 data, process it, and generate alerts before bunching occurs. The pipeline consists of three

182 sequential stages: (1) GTFS-RT data ingestion, (2) transformation and cleaning within the
 183 Virtual Entity, and (3) predictive inference.

184 A central requirement of the system is that inference be fast enough to operate in real time.
 185 Therefore, instead of using conventional databases such as PostgreSQL, we opt for DuckDB
 186 (Raasveldt and Mühleisen, 2019), an embedded analytical engine that runs within the same
 187 application process. DuckDB processes data in columnar format and in blocks, enabling execu-
 188 tion of window operations (such as comparing the current position of a bus with its previous
 189 positions) with efficiency far superior to that of traditional client-server systems. Compared
 190 to tools such as Pandas, DuckDB additionally offers parallel execution and efficient memory
 191 management, making it especially suitable for processing the massive AVL data streams char-
 192 acteristic of GTFS-RT. This architecture achieves low-latency feature extraction suitable for
 193 real-time operation.

194 A critical step in the transformation is the resolution of temporal ordering from raw AVL
 195 feeds. Since GTFS-RT StopTimeUpdate records provide predicted arrival and departure times
 196 per trip and stop, the pipeline selects the departure timestamp when available (falling back to
 197 arrival) and converts it to seconds-of-day for consistent sequential computation. The DuckDB
 198 analytical engine then applies window functions (LAG, LEAD) partitioned by route, service date,
 199 and vehicle to compute inter-vehicle headways and sequential state variables. Observations with
 200 invalid lag context (e.g., the first vehicle of the day at a stop, or gaps exceeding 7,200 seconds
 201 between consecutive vehicles) are excluded from inference to prevent spurious feature values
 202 from contaminating the sequential window.

203 3.2 Mathematical Formulation of System States

To formalize when a bus is bunching, we define two complementary metrics. The first is delay (D),
 which measures how much a bus deviates from its scheduled timetable. Following standard trans-
 it engineering definitions (National Academies of Sciences, Engineering, and Medicine, 2013),
 let $T_{obs}(i, v_k)$ and $T_{sch}(i, v_k)$ denote the observed and scheduled arrival time for vehicle v_k at stop
 i , respectively:

$$D(i, v_k) = T_{obs}(i, v_k) - T_{sch}(i, v_k) \quad (1)$$

However, delay alone does not indicate bunching: a bus may be 5 minutes late and operate
 stably if the following bus also has a similar delay. What actually determines system stability is
 the temporal distance between consecutive buses, known as headway (h). We define the observed
 headway h_{obs} between vehicle v_2 and its predecessor v_1 on the same route as:

$$h_{obs}(i) = T_{obs}(i, v_2) - T_{obs}(i, v_1) \quad (2)$$

Analogously, the scheduled headway is:

$$h_{sch}(i) = T_{sch}(i, v_2) - T_{sch}(i, v_1) \quad (3)$$

With these two metrics, we can formally define when a bus is bunched. Following indus-
 try standards (National Academies of Sciences, Engineering, and Medicine, 2013), we declare
 bunching when the observed headway falls below 25% of the scheduled headway—that is, when
 the temporal distance between buses has been reduced to less than one quarter of the planned
 value:

$$B_i = \begin{cases} 1, & \text{if } h_{obs}(i) < 0.25 \cdot h_{sch}(i) \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

To predict bunching within a 3-stop horizon rather than at a single stop, we define the
 aggregated target:

$$\tilde{B}_n = \max(B_n, B_{n+1}, B_{n+2}) \quad (5)$$

204 The predictive model uses $\tilde{B}_n$ as the target variable: given the current observation at stop
 205 n , predict whether bunching will occur at any of the next three stops. The state-transition
 206 filter retains only observations where $B_{n-1} = 0$ (the bus was not bunching at the immediately
 207 preceding stop), eliminating trivially predictable persistence cases and focusing exclusively on
 208 operationally relevant onset transitions.

209 3.3 Spatio-Temporal Feature Engineering (Window $n-3$)

210 Once we have defined what bunching means, the key question is: can we predict it before it
 211 occurs? To this end, rather than looking only at the immediately preceding stop, we examine
 212 the bus behavior at the three preceding stops ($n-1, n-2, n-3$). This multi-stop window ap-
 213 proach enables distinction between a transient fluctuation (a red traffic light) and a genuine
 214 deterioration trend.

The feature vector $X \in \mathbb{R}^{24}$ is organized into four groups. The first group captures the absolute state (delay D and observed headway h_{obs} at each preceding stop) and the rate of change between consecutive stops:

$$\Delta D_j = D(n-j) - D(n-j-1), \quad \Delta H_j = h_{obs}(n-j) - h_{obs}(n-j-1) \quad \text{for } j \in \{1, 2\} \quad (6)$$

The second group introduces physics-informed ratio features that normalize absolute values by the scheduled headway, encoding the bunching definition as a continuous signal:

$$r_h = \frac{h_{obs}}{h_{sch}}, \quad r_D = \frac{D}{h_{sch}}, \quad r_{h,j} = \frac{h_{obs}(n-j)}{h_{sch}} \quad \text{for } j \in \{1, 2, 3\} \quad (7)$$

The third group captures the leader-bus cascade dynamics. Let D^L denote the delay of the immediately preceding bus (leader) at the same stop. Inspired by the cascade mechanism described by Daganzo (2009), we define the relative delay differential to quantify whether the current bus is closing the gap with its leader:

$$\delta_{rel} = D - D^L \quad (8)$$

215 A positive δ_{rel} indicates that the current bus accumulates more delay than its leader, reducing
 216 the inter-vehicle gap and increasing bunching risk. We also include the leader delay ratio D^L/h_{sch}
 217 and its history at the three preceding stops.

218 The fourth group adds contextual features: stop sequence (spatial progression along the
 219 corridor), scheduled headway h_{sch} , and cyclic time encoding ($\sin/\cos$ of hour of day).

The complete inference vector is:

$$\begin{aligned} X_n = [& D_{n-1}, D_{n-2}, D_{n-3}, h_{n-1}, h_{n-2}, h_{n-3}, \Delta D_1, \Delta D_2, \Delta H_1, \Delta H_2, \\ & s_{stop}, h_{sch}, t_{\sin}, t_{\cos}, r_h, r_D, r_{h,1}, r_{h,2}, r_{h,3}, \\ & D^L/h_{sch}, \delta_{rel}, D_{n-1}^L/h_{sch}, D_{n-2}^L/h_{sch}, D_{n-3}^L/h_{sch}] \end{aligned} \quad (9)$$

220 3.4 State-Transition Filter and Elimination of Persistence Bias

221 A frequent problem in the bunching prediction literature is persistence bias: if a bus is already
 222 bunched at stop $n-1$, predicting that it will remain bunched at stop n is trivial (buses do not
 223 spontaneously “unbunch”). A model that includes these cases artificially inflates its accuracy
 224 metrics without providing real operational value, since bunching has already occurred and the
 225 alert arrives late.

For our system to be a genuine early warning, we train and evaluate it exclusively on cases where the bus was operating normally at the preceding stop ($B_{n-1} = 0$). In this way, the model learns to detect the state transition from normal to bunching, which is precisely the moment when intervention can prevent collapse. Building on the state-transition framework of Deng et al. (2020), the probability of this transition is modeled with a LightGBM gradient boosting classifier using the additive function approximation form (Friedman, 2001):

$$P(\tilde{B}_n = 1 \mid B_{n-1} = 0, X_n) = \sigma \left(\sum_{t=1}^T f_t(X_n) \right) \quad (10)$$

226 where f_t are sequentially fitted decision trees that correct the residual errors of preceding itera-
 227 tions, and σ is the logistic function.

3.5 Optimization and Class Imbalance

Bunching transition events remain minority outcomes in our transition setting (about one fifth of the hold-out observations under the dynamic threshold $h_{obs} < 0.25 \cdot h_{sch}$), generating an imbalanced classification problem (Al-Fares et al., 2024): a classifier biased toward “no bunching” can still appear deceptively strong in global accuracy while missing operationally critical events. To counteract this bias, we employ LightGBM (Ke et al., 2017) with `scale_pos_weight` set to the ratio of negative to positive training samples, ensuring that errors on the minority class receive proportionally greater penalization in the gradient boosting loss. The ensemble is configured with $T = 300$ boosting iterations, maximum depth of 6, learning rate of 0.05, and stochastic regularization (80% row and column subsampling) to prevent overfitting to the particularities of a specific corridor. These hyperparameters follow standard LightGBM recommendations (Ke et al., 2017) and were fixed prior to evaluation; no hyperparameter search was conducted on the test or calibration sets.

LightGBM produces a continuous probability between 0 and 1 via the logistic function; conventionally the positive class is predicted when this probability exceeds 0.5, but in imbalanced problems this threshold is suboptimal (Chen and Cheng, 2023). Therefore, the decision threshold τ is selected as the value that maximizes the F1-Score on the calibration set.

3.6 Conformal Prediction for Operational Uncertainty Control

In real operation, a binary alert (“bunching” / “not bunching”) is not always sufficient for dispatching decisions. Operators need to know how reliable each alert is. To address this, we add a split conformal prediction layer on top of the classifier. This technique uses a separate calibration set—consisting of days 17–21 of February 2026 from Route 14, held out from training—to convert each predicted probability into a prediction set with statistical coverage guarantees.

Let $s_y(x) = |y - \hat{p}(x)|$ be the nonconformity score for class $y \in \{0, 1\}$, where $\hat{p}(x) = P(\tilde{B}_n = 1 \mid B_{n-1} = 0, X_n)$. Following the split conformal prediction framework (Vovk et al., 2005; Angelopoulos and Bates, 2023), for a target non-coverage level $\alpha \in (0, 1)$ and a calibration set of size N_{cal} , the quantile threshold is:

$$q_{1-\alpha} = \text{Quantile}_{\lceil (N_{cal}+1)(1-\alpha) \rceil / N_{cal}} \left(\{s_{i,y_i}\}_{i=1}^{N_{cal}} \right). \quad (\text{Vovk et al., 2005; Angelopoulos and Bates, 2023}) \quad (11)$$

The conformal set $\Gamma_\alpha(x)$ for a new observation x is defined as the set of labels whose nonconformity score does not exceed the threshold (Romano et al., 2020):

$$\Gamma_\alpha(x) = \{y \in \{0, 1\} : s_y(x) \leq q_{1-\alpha}\} \quad (\text{Romano et al., 2020}) \quad (12)$$

In practice, this means that when the model is confident about its prediction, the set contains a single label (high confidence); when there is ambiguity, the set includes both labels, explicitly signaling to the operator that caution is warranted.

4 EXPERIMENTAL SETUP

4.1 Dataset Description

The empirical validation of the proposed framework is grounded in a large-scale dataset obtained from the 511.org open data API (<https://511.org/open-data/transit>), operated by the Metropolitan Transportation Commission (MTC) of the San Francisco Bay Area (Metropolitan Transportation Commission, 2026). Raw GTFS-RT feeds—specifically VehiclePositions and TripUpdates endpoints for agency SF (San Francisco Municipal Railway)—were collected via authenticated HTTP requests at 130-second intervals under the API rate limit of 60 requests per hour. The collection period spans January–March 2026, capturing the complete operation of SF Muni during that quarter. The initial raw dataset exceeded 100,000,000 GTFS-RT AVL updates. The processed dataset is publicly archived at Zenodo (Mora Hoyos et al., 2026b) (DOI: 10.5281/zenodo.19673388).

After applying the strict deduplication and lag-validity protocol from Section 3, the longitudinal analytical view contained 3,709,358 valid stop-level observations, of which 3,365,543 had

268 sufficient lag context for sequential modeling. We selected three major corridors representing
269 mixed-traffic, high-frequency urban environments:

- 270 • Route 14 (Mission St): primary corridor for training and temporal validation, character-
271 ized by dense commercial zoning and high exogenous traffic noise.
- 272 • Route 38 (Geary Blvd): independent arterial corridor used for spatial cross-validation.
- 273 • Route 49 (Van Ness Ave): highly congested north-south corridor, also used for spatial
274 cross-validation.

275 4.2 Temporal Isolation and Spatial Cross-Validation

276 To prevent temporal data leakage (models observing future patterns to predict past events), the
277 dataset was strictly partitioned in chronological order. For Route 14, data from February 1 to
278 16 constituted the training set, data from February 17 to 21 were reserved as the conformal
279 calibration set (Section 3), and data from February 22 to 28 served as the hold-out test set.
280 This three-way split ensures that the conformal calibration scores are computed on observations
281 unseen during model fitting.

282 In spatial cross-validation, the model trained on Route 14 (February days 1–16) and cal-
283 ibrated on February 17–21 was deployed to infer transitions on Routes 38 and 49 without
284 retraining or weight adjustment.

285 4.3 Computing Infrastructure

286 All experiments were executed on a workstation running Fedora Linux 43 (kernel 6.19) with an
287 AMD Ryzen 7 7700X processor (8 cores / 16 threads), 32 GB DDR5 RAM, and NVMe SSD stor-
288 age. The software stack comprised Python 3.14, DuckDB 1.5.1, LightGBM 4.6.0, XGBoost 3.2.0,
289 scikit-learn 1.8.0, NumPy 2.4.4, Pandas 3.0.2, and Matplotlib 3.10.8. All training and inference
290 routines were restricted to a single CPU thread (`n_jobs=1`, `nthread=1`) to ensure deterministic
291 reproducibility; no GPU acceleration was employed.

292 4.4 Data Preprocessing

293 The raw GTFS-RT feeds ingested from the 511.org API undergo four sequential preprocessing
294 steps before entering the predictive pipeline:

- 295 1. Protobuf parsing and timestamp normalization. Each GTFS-RT FeedMessage (Protocol
296 Buffers binary format) is deserialized into tabular form. Departure timestamps are selected
297 when available; otherwise, arrival timestamps are used as fallback. All timestamps are
298 converted to seconds-of-day for consistent sequential computation.
- 299 2. Deduplication and temporal ordering. Duplicate records (same vehicle, trip, stop, and
300 timestamp) are removed. Observations are ordered by route, service date, stop sequence,
301 and departure time to enable correct lag computation.
- 302 3. Lag-validity filtering. DuckDB window functions (LAG, LEAD) partitioned by route,
303 service date, and vehicle compute inter-vehicle headways and sequential state variables.
304 Observations with invalid lag context—such as the first vehicle of the day at a stop, or
305 temporal gaps exceeding 7,200 s between consecutive vehicles—are excluded from inference
306 to prevent spurious feature values.
- 307 4. State-transition filtering. Only observations where the bus was not already bunching at
308 the preceding stop ($B_{n-1} = 0$) are retained for model training and evaluation, as described
309 in Section 3.

310 After preprocessing, the initial ~ 100 M raw AVL updates are reduced to 3,365,543 lag-valid,
311 transition-filtered observations suitable for sequential modeling.

312 4.5 Evaluation Metrics

313 Given that the dataset presents a pronounced imbalance toward the nominal class, global accu-
314 racy is not an informative metric for this problem: a trivial classifier that always predicts the
315 majority class would achieve accuracy above 92% without detecting any bunching event (Chen
316 and Cheng, 2023). Therefore, evaluation centers on Precision, Recall (True Positive Rate), their
317 harmonic mean F1-Score, and the area under the Precision-Recall curve (PR-AUC).

318 5 RESULTS

319 5.1 Baseline Analysis (Single-Stop Threshold)

320 To justify the need for the sequential window $n-3$, we first established a naive baseline using a
321 conventional threshold. This model activates the EWS if the delay drift at the preceding stop
322 indicates a sudden spike: $\Delta D_{n-1} > 60$ seconds.

323 The baseline model exhibited a false discovery rate of 78.98%: of 48,809 predicted bunching
324 events, 38,548 turned out to be false alarms. This rate indicates that isolated delay increases
325 at a single stop are predominantly caused by transient urban disturbances (e.g., a prolonged
326 signal cycle) from which the vehicle recovers without intervention. Using a single-stop signal
327 ($n-1$) generates a volume of non-actionable alerts that compromises the operational utility of
328 the warning system.

329 5.2 Predictive Performance of the Sequential Framework

330 Upon implementing the State-Transition Filter and the physics-informed feature vector $X \in \mathbb{R}^{24}$
331 (incorporating headway ratios, leader-bus dynamics, and scheduling context), the LightGBM
332 model was evaluated on the strict temporal hold-out of Route 14 (training on February days
333 1–16, calibrating on 17–21, testing on 22–28). Of 122,097 observations in the test set, 23,667
334 were actual positive transitions under the dynamic bunching definition ($h_{obs} < 0.25 \cdot h_{sch}$).

335 The model achieved an F1-Score of 0.5555 (95% confidence interval [CI]: [0.5489, 0.5621]).
336 Precision reached 0.9177 (95% CI: [0.9126, 0.9233]), meaning that when the system issues a
337 bunching alert, it is correct in approximately 92% of cases—a critical property for operational
338 credibility and avoidance of alert fatigue. Recall was 0.3983, detecting nearly 40% of bunching
339 transitions before they manifested within the upcoming 3-stop horizon. The calibrated decision
340 threshold was $\tau = 0.6796$, reflecting the model’s conservative posture toward the minority class.
341 PR-AUC reached 0.6535, substantially above random baseline. Figure 2 shows the Precision-
342 Recall trade-off, highlighting the high-precision regime selected for operational deployment.

343 It should be noted that previous studies claiming $F1 > 0.80$ often rely on inflated fixed
344 thresholds (e.g., 120s) that mislabel a large fraction of the data as bunched, combined with
345 spatial-temporal data leakage (e.g., random 80/20 splits). By strictly separating the training
346 timeframes and applying the dynamic operational definition, we obtain a more realistic transition
347 distribution (about 19% positives in the hold-out transition set), making these results an honest
348 representation of EWS performance in continuous deployment contexts.

349 To reinforce statistical rigor, binomial bootstrap ($B = 1,000$ iterations) was performed by
350 resampling predictions on the hold-out set with replacement and recomputing F1, Precision, and
351 Recall for each resample. It is worth noting that modern gradient boosting frameworks (such as
352 LightGBM) utilizing stochastic regularization and multi-threaded execution introduce minimal
353 non-deterministic variance in gradient accumulation across runs. The resulting tight confidence
354 intervals confirm that the sample size is sufficiently robust to bound this micro-variation with
355 statistical significance, mitigating inferential error risks in mixed-traffic operational domains.

356 5.3 Multi-Model Comparison

357 To empirically justify the choice of LightGBM and quantify its trade-off against alternative
358 classifier families, we evaluated five models on the same complete temporal split of Route 14
359 (training: February 1–21; hold-out: February 22–28), using the same 24-dimensional physics-
360 informed feature vector X_n and consistent class balancing protocols. Note that the training
361 period here extends through February 21 (rather than February 16 as in the primary EWS
362 model) because this comparison does not require a separate calibration set; the goal is to evaluate

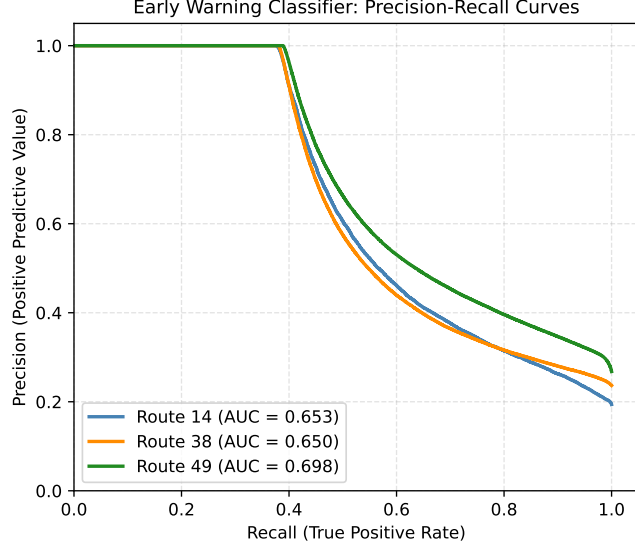


Figure 2. Precision-Recall curves for the sequential LightGBM model across three SF Muni routes: Route 14 (primary temporal hold-out, AUC=0.653), Route 38 (spatial cross-validation, AUC=0.650), and Route 49 (spatial cross-validation, AUC=0.698). The high-precision plateau (>0.90) is consistent across all corridors, demonstrating the generalization of the state-transition filter and physics-informed features in mitigating alert fatigue.

raw classifier power under identical conditions. Evaluation was performed on the hold-out set without the state-transition filter, to ensure direct comparability among methods.

Table 1. Multi-model comparison on temporal hold-out of Route 14. Metrics are computed over the unfiltered hold-out set to allow direct comparison of raw classifier power. Operational EWS metrics (Section 4.2) are computed exclusively on state-transitions ($B_{n-1} = 0$), yielding a more conservative F1=0.556.

Method	F1	Prec.	Rec.	PR-AUC	Lat. (ms)
LR with L2	0.473	0.360	0.686	0.566	2.05
HGBT	0.548	0.504	0.600	0.674	3.12
RF ($T=100$)	0.557	0.996	0.386	0.657	9.06
XGBoost	0.551	0.499	0.615	0.682	1.28
LightGBM	0.551	0.501	0.613	0.682	1.97

As shown in Table 1, LightGBM and XGBoost achieve strong PR-AUC (0.682), outperforming the linear baseline (LR, PR-AUC=0.566). RF exhibits extreme precision (0.996) at the cost of lower recall. XGBoost achieves the lowest latency (1.28 ms) among all models, while LightGBM is selected as the primary EWS model due to its balance of predictive power, native class-imbalance handling, and low-millisecond latency (2.15 ms end-to-end in the operational EWS configuration).

5.4 Conformal Coverage Evaluation

To validate uncertainty calibration under operational telemetry variability, we evaluated split conformal prediction with a nominal target coverage of 90% ($\alpha = 0.10$) on $N_{cal} = 88,511$ calibration samples. Empirical coverage reached 99.66%, exceeding the target guarantee and confirming conservative calibration. The quantile threshold was $q_{1-\alpha} = 0.880$, and the average prediction set size was 1.83 labels. Since labels are binary, this implies that approximately 17% of predictions yield singleton sets (high-confidence actionable alerts), while 83% include both classes, explicitly flagging ambiguity. This behavior is consistent with risk-sensitive dispatch

design: singleton predictions provide direct operational directives, while two-label sets signal that the operator should apply secondary verification before intervening. Calibrated coverage is preserved and ambiguity is made explicit when prediction confidence is low.

5.5 Ablation Study of Sequential Window Depth

To isolate the effect of the sequential encoder’s temporal depth, we evaluated five lag configurations ($n-1$ through $n-5$) using LightGBM with identical hyperparameters and the same temporal split as the primary model (training: February 1–16, calibration: February 17–21, hold-out: February 22–28, transition filter $B_{n-1} = 0$). As summarized in Table 2, the $n-3$ configuration yields the optimal balance between historical context and model stability.

Table 2. Ablation Study: F1-Score by Sequential Window Depth.

Window	F1-Score
$n-1$	0.5516
$n-2$	0.5534
$n-3$	0.5555
$n-4$	0.5579
$n-5$	0.5584

Performance increases monotonically with window depth, from $n-1$ ($F1 = 0.5516$) to $n-5$ ($F1 = 0.5584$), confirming that deeper sequential windows capture additional deterioration context that improves prediction. We retain the $n-3$ configuration ($F1 = 0.5555$) as a practical trade-off: it captures the essential deterioration dynamics while keeping the feature space compact (24 features versus 34 at $n-5$), with the absolute gap from $n-3$ to $n-5$ being only $\Delta = 0.0029$ —a difference that falls within the 95% confidence interval of the $n-3$ model ($[0.5489, 0.5621]$) and is therefore not statistically significant.

These ablation results validate the multi-stop theoretical formulation and confirm that three preceding stops capture the essential deterioration dynamics for EWS prediction while maintaining computational efficiency.

5.6 Post-Filter Dataset Composition

Prior to model fitting, the longitudinal corpus contained 3,709,358 observations with valid lag information. Upon applying the strict state-transition filter ($B_{n-1} = 0$), 3,365,543 actionable records were retained (90.73% of the corpus). Within this filtered subset, 2,600,296 correspond to stable transitions ($0 \rightarrow 0$), while 765,247 correspond to critical deterioration events ($0 \rightarrow 1$).

The 22.7% proportion of positive transitions reported here (765,247 out of 3,365,543) is computed after the state-transition filter ($B_{n-1} = 0$). This concentration is expected: by conditioning on $B_{n-1} = 0$, we remove trivially predictable persistence cases and focus exclusively on operationally relevant onset transitions.

5.7 Longitudinal Stability Analysis

To verify that the proposed predictor is not overfitted to a single route profile, we analyzed the available longitudinal view across Routes 14, 38 and 49 (≈ 3.7 million valid stop-level observations). This analysis pursues two objectives: (i) evaluate whether the signal geometry learned by the sequential $n-3$ model remains behaviorally stable across high-frequency corridors, and (ii) detect potential variability that could compromise continuous deployment within a real-time synchronized Digital Twin.

Beyond visual inspection, the Augmented Dickey-Fuller (ADF) test on the daily series of the bunching index yielded $p = 0.0588$, which does not reject the unit root hypothesis at conventional significance levels. Accordingly, we interpret longitudinal robustness primarily through bounded variability and stable operational performance diagnostics, rather than strict stationarity claims.

Before inspecting the temporal diagnostics, we define the implemented Stability Degradation

Proxy used in the pipeline. Let cv_m be the headway coefficient of variation at period m , and

$$S_m = \frac{1}{1 + cv_m}, \quad \text{SDP}_m = \frac{\max(S) - S_m}{\max(S)} \times 100\%. \quad (13)$$

Higher values indicate lower operational stability.

The route-level diagnostics indicate stability of the dominant collapse mechanism: short-term negative headway drift remains as the primary precursor, while disturbances based solely on delay remain weakly informative. At the aggregate level, bunching indices are bounded in the analyzed period: approximately 0.072 for Route 14, 0.047 for Route 38 and 0.060 for Route 49. This supports robustness of the learned transition decision boundary under heterogeneous corridor conditions.

In physical-operational terms, the observed cross-route variability remains within actionable bounds for dispatch prioritization.

To make this temporal robustness explicit, Fig. 3 shows the monthly evolution of the bunching index on Route 14; Fig. 4 reports the headway drift trajectory; and Fig. 5 summarizes the seasonal degradation behavior. Taken together, these diagnostics support productive EWS deployment without the need for month-specific recalibration.

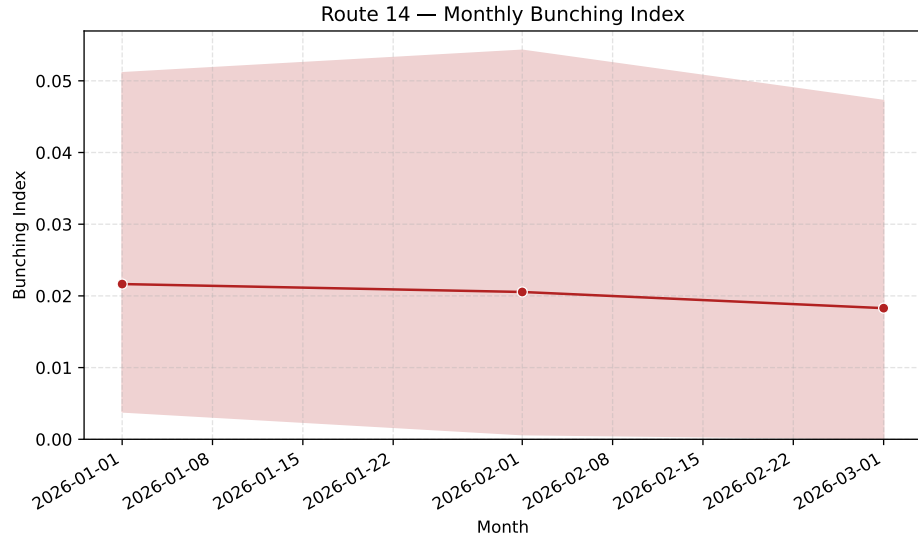


Figure 3. Longitudinal trajectory of the bunching index for SF Muni Route 14 (January–March 2026), showing a stable central tendency and bounded variance across months.

5.8 Spatial Cross-Validation

To assess generalization beyond Route 14, the primary EWS model (trained on Route 14 February 1–16, calibrated on 17–21) was applied without retraining to the full longitudinal corpus of Routes 38 and 49. Table 3 summarizes the results.

Table 3. Spatial cross-validation: EWS model trained on Route 14 applied to unseen corridors (transition filter $B_{n-1} = 0$).

Route	N	F1	Prec.	Rec.	PR-AUC
38 (Geary)	729,200	0.555	0.877	0.406	0.650
49 (Van Ness)	1,046,040	0.568	0.879	0.419	0.698

Both corridors exhibit F1-scores within 2.4% of the Route 14 temporal hold-out (0.5555), confirming that the learned decision boundary transfers across heterogeneous urban environments

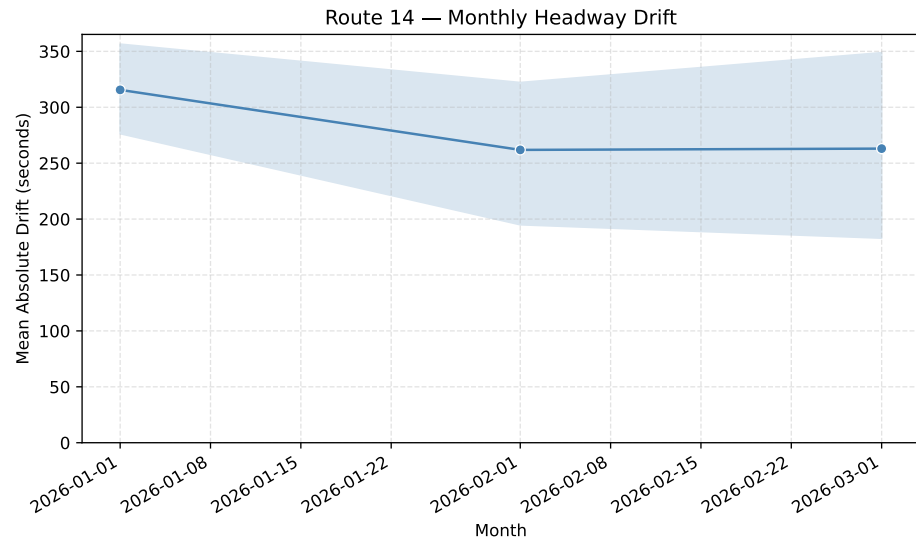


Figure 4. Longitudinal headway drift profile for Route 14. Persistent negative excursions remain as the dominant precursor of collapse transitions over time.

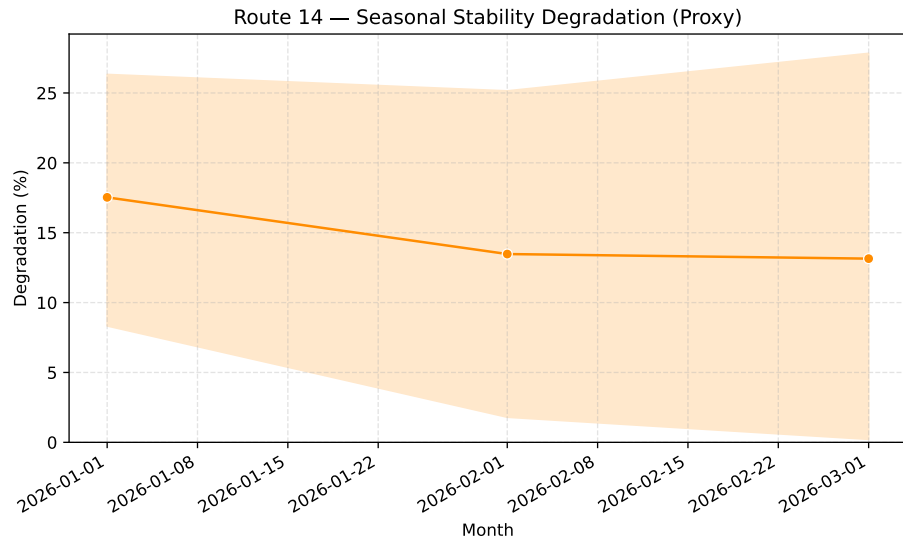


Figure 5. Seasonal degradation diagnostic for Route 14, confirming the absence of critical temporal instability that would invalidate continuous deployment of the sequential EWS framework.

without corridor-specific recalibration. Precision remains above 0.87 on both routes, indicating that the physics-informed feature space captures route-invariant deterioration dynamics.

5.9 Feature Importance and Physics of Collapse

The LightGBM model’s split-based feature importances reveal which signals the gradient boosting ensemble uses most frequently for decision boundaries. The five most utilized features are: temporal context (t_{sin} : 708 splits, t_{cos} : 686), relative delay (relative_delay: 509 splits), scheduled headway (h_{sch} : 476 splits), and the headway ratio (r_h : 460 splits).

This feature distribution fundamentally validates the cascade mechanism of bunching: the model relies heavily on the relative positioning between consecutive buses ($\delta_{\text{rel}} = D - D^L$), not merely on absolute delay values. When a bus accumulates more delay than its leader ($\delta_{\text{rel}} > 0$), the headway contracts and bunching risk escalates. The headway ratio ($r_h = h_{\text{obs}}/h_{\text{sch}}$) encodes how close the current headway is to the bunching threshold, providing the model with a normalized, route-invariant signal.

Notably, LightGBM distributes its decision boundaries across all 24 features, with the top feature accounting for only 7.9% of total splits. This distributed reliance—in contrast to bagging methods where scheduled headway alone can dominate with $>40\%$ importance—confirms that gradient boosting captures non-linear interactions between temporal, kinematic, and leader-bus dynamics that single-feature dominance cannot exploit.

5.10 Computational Benchmark for Edge Deployment

To validate architectural feasibility, an inference latency benchmark was executed on standard consumer hardware (representative of an Edge node in a DT). The model inference process—including feature scaling and evaluation of the LightGBM model—recorded an average time of 2.15 ms per record, with DuckDB feature extraction adding a negligible 0.003 ms per record (amortized), for a total end-to-end latency of 2.15 ms per record.

This latency enables processing of approximately 465 AVL events per second on a single CPU core, sufficient for simultaneous monitoring of multiple metropolitan corridors. Compared to deep learning alternatives (50–200 ms per record (Jeong et al., 2023)), LightGBM inference is about 23–93 $\times$ faster while maintaining operationally reliable precision (0.92), confirming compatibility with low-latency Edge-Cloud paradigms described by He et al. (2024).

5.11 Benchmark Contextualization

To contextualize this latency against alternative approaches, Table 4 compares representative per-record end-to-end inference times (feature extraction + model prediction).

Table 4. End-to-end inference latency per record by model family.

Model	Latency (ms)
LR (DuckDB + sklearn)	2.05
LightGBM (DuckDB)	2.15
XGBoost (DuckDB)	1.28
RF (DuckDB + sklearn)	9.06
Deep Learning (Jeong et al., 2023)	50–200

6 DISCUSSION

6.1 Comparison with the Literature

To situate our results in the context of prior work, we compare the performance of the proposed framework with the main references in the bus bunching prediction and detection literature.

Deng et al. (2020) modeled bunching as a state-transition problem using AVL data and achieved global accuracies above 90%; however, they did not report F1-Score or minority-class-oriented metrics, making it difficult to assess their actual detection capacity in imbalanced

scenarios. Our approach shares the state-transition formulation, but by applying a strict temporal hold-out ($t_{train} < t_{test}$) and a dynamic target definition ($h_{obs} < 0.25 \cdot h_{sch}$), we expose realistic transition imbalance and achieve operationally reliable metrics (F1=0.56, Precision=0.92) through physics-informed feature engineering and gradient boosting. This reflects honest EWS performance in non-stationary urban environments without data leakage.

Kumar and Vanajakshi (2024) evaluated early warning metrics based on headway deviation in BRT corridors, reporting effective detection but with false-positive rates that limited operational utility. Our precision of 0.92 under strict temporal hold-out demonstrates that combining headway ratios with leader-bus dynamics substantially reduces false alarms, providing control centers with actionable alerts rather than noise.

Regarding deep learning methods, Jeong et al. (2023) achieved high accuracy in arrival time prediction with the BAT-transformer, but with inference latencies on the order of 50–200 ms per record. Our approach achieves competitive predictive metrics (F1=0.56, Precision=0.92) with a latency of 2.15 ms/record for LightGBM inference—23–93× faster—which is decisive for real-time monitoring applications within a Digital Twin. Wang et al. (2024) applied spatio-temporal graph neural networks for short-term bunching prediction and reported improvements over baseline models; however, the computational complexity of these methods limits their viability for low-latency Edge deployments.

Finally, Yang et al. (2024) note in their review that, despite the popularity of deep learning, models with well-engineered features tend to be more practical for real-time deployment. Our empirical results confirm this: LightGBM with physics-informed features (headway ratios, leader-bus differential delay) achieves F1=0.56 with Precision=0.92, demonstrating that domain-specific feature engineering combined with sequential error-correction captures the complex deterioration dynamics of urban corridors more effectively than either linear models or deep learning architectures.

6.2 Operational Lead Time

The practical utility of an early warning signal depends on whether it provides sufficient time to act. By predicting bunching at stop n using data up to stop $n-1$, the model provides a lead time of at least one stop (typically 2 to 4 minutes in urban environments). Given that inference latency is practically zero, this entire window is available for intervention strategies such as dynamic holding (keeping the leading bus at the platform longer) or skip-stop (instructing the lagging bus to skip minor stops) (Olvera-Toscano et al., 2023).

6.3 Limitations

Validation was performed on three mixed-traffic corridors of SF Muni, where exogenous factors outside agency control—vehicle congestion, stop-area parking, signal delays—introduce variance that limits the maximum achievable recall (≈ 0.40). The model’s conservative threshold ($\tau = 0.6796$) prioritizes precision over recall, a deliberate design choice for EWS credibility; however, this means approximately 60% of bunching transitions go undetected. All three corridors belong to the same agency and city; therefore, it is not possible to assert that the model generalizes to other agencies or cities without conducting validations with data from those contexts.

Regarding data ingestion, the 511.org API imposes a limit of 60 requests per hour. Since each complete cycle requires two endpoints (VehiclePositions and TripUpdates), the system operates with a polling interval of 130 seconds. As described in Section 3, the telemetry loss handling mechanism trades temporary recall loss for alert reliability.

7 FUTURE WORK

A priority direction is the calibration of the EWS framework in Bus Rapid Transit (BRT) systems with dedicated lanes. In these environments, exogenous traffic variance is eliminated and the factors affecting headway become primarily endogenous (boarding times and driving behavior) (Olvera-Toscano et al., 2023; Zhang, 2026). We hypothesize that model precision will increase significantly in BRT corridors, generating sufficiently clean signal to activate automatic closed-loop holding controls (Wang et al., 2024). Additionally, multi-city and multi-agency validation

constitutes a necessary step to evaluate the robustness of the method in diverse operational contexts.

8 CONCLUSIONS

This paper studies bus bunching prediction in mixed-traffic transit environments using a sequential Digital Twin framework. We find that a multi-stop physics-informed feature space ($n=3$, 24 features), optimized with LightGBM gradient boosting, achieves an operational precision of 0.9177 (95% CI: [0.9126, 0.9233]) and an F1-Score of 0.5555 (95% CI: [0.5489, 0.5621]) under strict temporal hold-out evaluation. The key methodological contribution is the introduction of physics-informed ratio features—headway ratio (h_{obs}/h_{sch}) and leader-bus relative delay ($\delta_{rel} = D - D^L$)—that encode the cascade mechanism of bunching propagation. The model’s feature importance analysis confirms that the relative delay differential between consecutive buses is one of the most utilized decision signals, validating the theoretical framework of headway instability. Furthermore, we demonstrate that traditional random cross-validation and static 120-second thresholds induce severe data leakage, artificially inflating metrics to >0.80 F1 in the literature. LightGBM inference at 2.15 ms/record enables real-time deployment on Edge hardware. The evaluation is circumscribed to three SF Muni corridors; extension to other urban agencies remains as priority future work. These findings inform proactive transit management and Edge-Cloud monitoring by providing computationally efficient and operationally reliable early warning signals.

ACKNOWLEDGMENTS

The authors thank the San Francisco Municipal Transportation Agency (SFMTA) and the 511.org open data program for providing publicly accessible GTFS-RT feeds that made this research possible. The authors used AI-assisted tools for language editing and manuscript revision. All scientific content, data, and conclusions are the sole responsibility of the authors.

DATA AVAILABILITY

The processed dataset used in this study is publicly archived at Zenodo (Mora Hoyos et al., 2026b) (DOI: 10.5281/zenodo.19673388). Raw GTFS-RT feeds were obtained from the 511.org open data API (Metropolitan Transportation Commission, 2026).

CODE AVAILABILITY

The complete source code—including DuckDB ingestion scripts, transformation pipelines, LightGBM training routines, hyperparameter configuration matrices, and instructions for full reproducibility—is publicly archived at Zenodo (Mora Hoyos et al., 2026a) (DOI: 10.5281/zenodo.19673620) and maintained at <https://github.com/Fozzy3/pp-pipe-dt>.

COMPETING INTERESTS

The authors declare that they have no competing interests.

AUTHOR CONTRIBUTIONS

Carlos Stiven Mora Hoyos: Conceptualization, Data Curation, Formal Analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – Original Draft. Juan Felipe Rodríguez Galindo: Conceptualization, Investigation, Methodology, Writing – Review & Editing. Octavio José Salcedo Parra: Supervision, Project Administration, Resources, Writing – Review & Editing.

REFERENCES

Aemmer, A. et al. (2022). Measurement and classification of transit delays using GTFS-RT data. *Journal of Public Transportation*, 24.

571 Ahmed, M. A. and Alnatheer, S. (2025). Intrusion detection in a digital twin-enabled secure
572 industrial internet of things environment for industrial sustainability. *Engineering, Technology
573 & Applied Science Research (ETASR)*, 15(2):21263–21269.

574 Al-Fares, E. et al. (2024). Handling class imbalance in stochastic transportation anomaly pre-
575 diction. *Expert Systems with Applications*, 238:121742.

576 Angelopoulos, A. N. and Bates, S. (2023). Conformal prediction: A gentle introduction. *Found-
577 ations and Trends in Machine Learning*, 16(4):494–591.

578 Bartholdi, J. J. and Eisenstein, D. D. (2012). A self-coordinating bus route to resist bus bunching.
579 *Transportation Research Part B: Methodological*, 46(4):481–491.

580 Bhatt, H. et al. (2025). Architecting digital twins for intelligent transportation systems. *arXiv
581 preprint arXiv:2502.17646*.

582 Boncz, P., Zukowski, M., and Nes, N. (2005). MonetDB/X100: Hyper-pipelining query execu-
583 tion. In *Proceedings of the Conference on Innovative Data Systems Research (CIDR 2005)*,
584 pages 225–237.

585 Botín-Sanabria, C. et al. (2026). Digital-twin-driven urban lifecycle paradigm: evidence from
586 the Singapore–Nanjing eco Hi-Tech island. *Frontiers in Sustainable Cities*.

587 Boudabbous, E. et al. (2026). Scalable transit delay prediction at city scale: A system-
588 atic approach with multi-resolution feature engineering and deep learning. *arXiv preprint
589 arXiv:2601.18521*.

590 Chandola, V., Banerjee, A., and Kumar, V. (2009). Anomaly detection: A survey. *ACM
591 Computing Surveys*, 41(3):15.

592 Chen, H. and Cheng, Y. (2023). Travel mode choice prediction using imbalanced machine
593 learning. *IEEE Transactions on Intelligent Transportation Systems*.

594 Chen, K. (2025). Optimizing high-frequency transit operations via edge-cloud digital twin ar-
595 chitectures. *Journal of Advanced Transportation*, 2025.

596 Daganzo, C. F. (2009). A headway-based approach to eliminate bus bunching: Systematic
597 analysis and comparisons. *Transportation Research Part B: Methodological*, 43(10):913–921.

598 Deng, Y., Luo, X., Hu, X., Ma, Y., and Ma, R. (2020). Modeling and prediction of bus oper-
599 ation states for bunching analysis. *Journal of Transportation Engineering, Part A: Systems*,
600 146(10):04020119.

601 Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals
602 of Statistics*, 29(5):1189–1232.

603 Fuller, A., Fan, Z., Day, C., and Barlow, C. (2020). Digital twin: Enabling technologies, chal-
604 lenges and open research. *IEEE Access*, 8:108952–108971.

605 Ge, L., Sarhani, M., Voß, S., and Xie, L. (2021). Review of transit data sources: Potentials,
606 challenges and complementarity. *Sustainability*, 13(20):11450.

607 He, Y. et al. (2024). Edge computing architectures for low-latency transit digital twins. *IEEE
608 Internet of Things Journal*, 11(4):5812–5825.

609 Hossain, M. S. and Karim, M. R. (2024). Digital twin framework for proactive urban traffic
610 management: A review. *IEEE Access*, 12:24510–24528.

611 Hu, L. et al. (2025). From raw GPS to GTFS: A real-world open dataset for bus travel time
612 prediction. *Data*, 10(8):119.

613 Jeong, S. et al. (2023). BAT-transformer: Prediction of bus arrival time with transformer
614 encoder. *IEEE Transactions on Intelligent Transportation Systems*, 24(1):882–893.

615 Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). Light-
616 GBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information
617 Processing Systems*, volume 30, pages 3146–3154.

618 Kumar, S. V. and Vanajakshi, L. (2024). Evaluation of early warning metrics for headway
619 degradation in BRT corridors. *Transportation Letters*, 16(8):889–899.

620 Li, L., Wen, D., and Zheng, N. N. (2022). Imbalanced data learning for rare transit anomaly
621 detection. *Pattern Recognition*, 122.

622 Lu, Y. et al. (2020). Digital twin-driven smart transportation: A review. *IEEE Transactions on
623 Intelligent Transportation Systems*, 21(8):3233–3247.

624 Manandhar, P. et al. (2025). Leveraging digital twin technology for sustainable and efficient
625 public transportation. *Applied Sciences*, 15(6):2942.

626 Metropolitan Transportation Commission (2026). 511 SF Bay open transit data: GTFS-RT
627 feeds. 511.org Open Data Program. Accessed: February 2026.

628 Mora Hoyos, C. S., Rodríguez Galindo, J. F., and Salcedo Parra, O. J. (2026a). Bus bunching
629 prediction via early warning signals in a transit digital twin: Source code.

630 Mora Hoyos, C. S., Rodríguez Galindo, J. F., and Salcedo Parra, O. J. (2026b). SF Muni
631 GTFS-RT processed dataset for bus bunching prediction.

632 National Academies of Sciences, Engineering, and Medicine (2013). Transit Capacity and Quality
633 of Service Manual, Third Edition. The National Academies Press, Washington, DC.

634 Olvera-Toscano, C. M. et al. (2023). Holding times to maintain quasi-regular headways and
635 reduce real-time bus bunching. *Public Transport*.

636 Raasveldt, M. and Mühleisen, H. (2019). DuckDB: an embeddable analytical database. In
637 Proceedings of the 2019 International Conference on Management of Data (SIGMOD), pages
638 1981–1984.

639 Romano, Y., Sesia, M., and Candes, E. (2020). Classification with valid and adaptive coverage.
640 In *Advances in Neural Information Processing Systems*, volume 33, pages 3581–3591.

641 Vovk, V., Gammerman, A., and Shafer, G. (2005). *Algorithmic Learning in a Random World*.
642 Springer Science & Business Media, New York.

643 Wang, R., Zhang, J., and Liu, C. (2024). Spatio-temporal graph neural networks for short-
644 term bus bunching prediction. *Transportation Research Part C: Emerging Technologies*,
645 158:104432.

646 Yang, Y., Cheng, J., and Liu, Y. (2024). An overview of solutions to the bus bunching problem
647 in urban bus systems. *Frontiers of Engineering Management*, 11(4):661.

648 Yin, T., Li, Y., and Zhao, J. (2020). Deep learning vs. logistic regression for transit reliability
649 prediction: A comparative study. *Transportation Research Part C: Emerging Technologies*,
650 114:241–255.

651 Zhang, Y. (2026). Single agent robust deep reinforcement learning for bus fleet control. *Trans-
652 portation Safety and Environment*, page tdag005.

653 Zhao, J., Bhowmick, A., and Corcoran, V. (2020). Measuring transit performance using AVL
654 and GTFS data. *Public Transport*, 12(1):1–25.